

Copy & Move Constructor

Junior / Mid C++ Հարցազրույցի խորացված ուղեցույց

Այս մասը ընդգրկում է Copy Constructor, Shallow vs Deep Copy, Move Constructor, `std::move`, lvalue/rvalue, lvalue reference (`&`) և rvalue reference (`&&`) թեմաները՝ memory-level բացատրություններով և պատրաստի interview պատասխաններով: Նպատակն է ոչ թե անգիր անել, այլ հասկանալ, թե memory-ում ինչ է կատարվում:

1. Copy Constructor

Ի՞նչ է Copy Constructor-ը

Սովորական constructor-ը կանչվում է object ստեղծելիս: Բայց երբ **object-ից նոր object ենք ստեղծում**, կանչվում է Copy Constructor-ը:

```
Person p1;  
Person p2 = p1;    // սա Copy Constructor է, ոչ սովորական constructor
```

Copy Constructor-ը special member function է, որը ստեղծում է նոր object՝ արդեն գոյություն ունեցող object-ի հիման վրա:

```
class Person {  
public:  
    string name;  
  
    Person(const Person& other) {  
        name = other.name;  
    }  
};
```

Syntax՝ `ClassName(const ClassName& other);`

Երբ է Copy Constructor-ը կանչվում

1. Object initialization

```
Person p2 = p1;    // կամ Person p2(p1);
```

2. Function parameter by value

```
void print(Person p) { }  
print(p1);    // Copy Constructor
```

3. Function return by value

```
Person create() {  
    Person p;  
    return p;    // հնարավոր է Copy Constructor (compiler-ը հաճախ optimize է անում)  
}
```

Compiler-ի Default Copy Constructor

Եթե դու Copy Constructor չգրես, compiler-ը ինքն է սարքում:

```
class Person {  
public:  
    string name;  
};  
  
Person p1;  p1.name = "Arman";  
Person p2 = p1;    // p2.name == "Arman"
```

Պարզ (POD/value) տվյալների դեպքում սա ճիշտ է աշխատում:

2. Shallow Copy vs Deep Copy

Խնդիրը սկսվում է pointer-ի դեպքում

```
class Test {  
public:  
    int* data;  
  
    Test() {  
        data = new int(10);  
    }  
};  
  
Test t1;  
Test t2 = t1;    // default copy -> t2.data = t1.data (միայն հասցեն)
```

Երկուսն էլ նույն memory-ն են ցույց տալիս.

```
t1.data ----+  
            |  
            v  
          [10]  
            ^  
            |  
t2.data ----+
```

Սա **Shallow Copy** է — միայն հասցեն (pointer-ը) է պատճենվում:

Shallow Copy-ի վտանգները

1. Shared state

```
*t2.data = 50;  
cout << *t1.data;    // 50 — քանի որ նույն memory-ն է
```

2. Double delete (ավելի վտանգավոր)

```

~Test() { delete data; }

// t2 -> delete data;
// t1 -> delete data;    (ևույն pointer-ը երկրորդ անգամ)
// -> DOUBLE DELETE -> Undefined Behavior -> Crash

```

Deep Copy լուծումը

Գրում ենք մեր սեփական Copy Constructor-ը, որը նոր memory է ստեղծում.

```

class Test {
public:
    int* data;

    Test() {
        data = new int(10);
    }
    Test(const Test& other) {
        data = new int(*other.data);    // deep copy
    }
    ~Test() {
        delete data;
    }
};

```

Հիմա.

```

t1.data ----> [10]
t2.data ----> [10]    (ևույն արժեք, տարբեր memory)

```

```

*t2.data = 500;
cout << *t1.data;    // 100-ը մնում է անփոփոխ

```

Interview պատասխան

Shallow Copy — Copies only pointers/addresses.

Deep Copy — Duplicates the actual resource (new allocation).

Compiler-generated copy constructor performs a shallow copy. Only the pointer value (address) is copied, so both objects point to the same memory. This can lead to shared state,

double deletion, and memory corruption.

3. Ինչու Copy Constructor-ը reference-ով է ընդունում

Սովորաբար գրում ենք `Test(const Test& other)`, ոչ թե `Test(Test other)`: Ինչո՞ւ
Եթե parameter-ը value լիներ (`Test other`), նրա մեջ `t1`-ը փոխանցելու համար պետք է նոր
copy ստեղծվեր, իսկ copy ստեղծելու համար պետք է Copy Constructor... և այսպես անվերջ.

Copy Constructor

└─ Copy Constructor

└─ Copy Constructor

└─ ... (infinite recursion)

Reference-ը copy չի ստեղծում, դրա համար՝ `const Test& other`: `const`-ը երաշխավորում է,
որ original object-ը չի փոխվի:

Reference-ը առանձին object չէ — այն պարզապես ուրիշ անուն է նույն variable-ի համար.

```
int x = 10;  
int& ref = x;    // ref ----> x (ևս չկա memory)
```

4. Move Constructor

Ինչո՞ւ է պետք

Պատկերացրո՛ւ շատ մեծ object.

```
vector<int> data(1000000);  
vector<int> data2 = data;    // copy -> 1,000,000 էլեմենտ պատճենել -> դանդաղ
```

Move-ի գաղափարը՝ **մի պատճենիր, ուղղակի տուր ownership-ը**:

Analogy. Copy = նույն մեքենայի լրիվ պատճեն սարքել: Move = բանալին տալ ուրիշին:
Մեքենան նույնն է, պարզապես տերը փոխվեց:

Move Constructor Syntax

```
ClassName(className&& other);    // && -> rvalue reference
```

```
class Test {  
public:  
    int* data;  
  
    Test() {  
        data = new int(10);  
    }  
    Test(Test&& other) {  
        data = other.data;    // pointer-ը տալիս ենք նոր object-ին  
        other.data = nullptr; // հին object-ը դատարկում ենք  
    }  
    ~Test() {  
        delete data;  
    }  
};
```

Memory-ում ինչ է լինում

Սկզբում.


```
t1 -----> [10]
```

Move-ից հետո.

```
t1 -----> nullptr  
t2 -----> [10]
```

Նոր memory չի ստեղծվել — ownership-ը փոխանցվել է:

5. std::move()

Ամենասիրած interview հարցերից: Շատերը կարծում են `std::move` object-ը տեղափոխում է — **իրականում ոչ**:

`std::move()` պարզապես type cast է անում, մոտավորապես `static_cast<T&&>(obj)`: Compiler-ին ասում է.

Այս object-ը կարելի է treat անել որպես rvalue (կարելի է move անել):

```
Test t1;
std::move(t1);           // ինքնին ոչինչ չի անում

Test t2 = std::move(t1); // ՍԱ արդեն Move Constructor է կանչում
```

Copy vs Move ամփոփ

Copy Constructor:	Move Constructor:
allocate new resource	steal resource pointer
copy resource	set old pointer to nullptr
	avoid expensive copy

	Copy Constructor	Move Constructor
Signature	<code>Test(const Test& other)</code>	<code>Test(Test&& other)</code>
Resource	նոր memory + copy	ownership transfer
Հին object	անփոփոխ	դատարկվում է (nullptr)
Արագություն	դանդաղ (deep copy)	արագ

6. lvalue և rvalue

lvalue

```
int x = 10;
```

`x`-ը **lvalue** է, քանի որ ունի անուն, հասցե, և կարող ես դիմել հետո:

```
cout << &x;    // աշխատում է
```

Օրինակներ՝ `x`, `y`, `person`, `arr[0]`:

rvalue

Ժամանակավոր արժեքներ, որոնք address չունեն.

```
10           // rvalue
x + 5        // rvalue
100          // rvalue
getValue()   // rvalue (եթե temporary object է վերադարձնում)
```

```
int x = 10;
int y = x + 5;
// x        -> lvalue
// x + 5    -> rvalue
```

Հիշելու հեշտ ձև՝ **lvalue = has location**, **rvalue = temporary value**:

7. lvalue reference (&) և rvalue reference (&&)

lvalue reference

```
int x = 10;
int& r = x;    // OK, bind է լինում lvalue-ի հետ

int& r2 = 10;  // ERROR, 10-ը rvalue է
```

rvalue reference

```
int&& r = 10;    // OK, 10-ը rvalue է
int&& r2 = x + 5; // OK
```

Ինչու ստեղծվեց rvalue reference-ը

Move Semantics-ի համար.

```
Test(Test&& other)    // other-ը rvalue reference է
```

Compiler-ը հասկանում է.

Այս object-ը ժամանակավոր է, շուտով կջնջվի, կարող ենք resources-ը «գողանալ»:

8. Reference Member ունեցող class

int& member

```
class Test {  
public:  
    int& ref;  
  
    Test(int& r) : ref(r) { }  
};  
  
int x = 100;  
Test t1(x);  
Test t2 = t1;    // default copy -> ref(other.ref)
```

Երկուսն էլ նույն variable-ն են նայում.

```
x = 100  
t1.ref ----> x  
t2.ref ----> x
```

```
t2.ref = 500;    // x -> 500, ուստի t1.ref-ն էլ 500
```

Սա սովորաբար **վտանգավոր չէ**, քանի որ reference-ը ownership չի պահում — ուղղակի alias է, resource management չկա:

Կարևոր առանձնահատկություն. reference member-ը reassignment չի կարող անել, դրա համար reference member ունեցող class-ների `operator=` -ը հաճախ խնդիրներ է առաջացնում:

The copy constructor copies the reference binding. Both objects will refer to the same original variable. No new object is created.

9. Ժամանակակից C++ լուծումը

Իրական նախագծերում հաճախ raw pointer չես գրի.

```
int* data;    // ձեռքով memory management, վտանգավոր
```

Փոխարենը՝

```
std::unique_ptr<int> data;  
std::vector<int> data;  
std::string data;
```

```
class Test {  
    std::vector<int> data;    // compiler-ի default copy/move-ն արդեն ճիշտ է  
};
```

Այստեղ compiler-ի default copy constructor-ը ճիշտ deep-copy է անում, և ձեռքով memory management պետք չէ (Rule of Zero):

Ամփոփ Cheat Sheet

Copy Constructor — `Test(const Test& other)` ; creates a new object as a copy of an existing one; deep copy for resources.

Shallow Copy — copies only the pointer address; risk of shared state and double delete.

Deep Copy — allocates new memory and duplicates the resource.

Move Constructor — `Test(Test&& other)` ; transfers ownership instead of copying; faster.

std::move — does NOT move; it casts to rvalue so the move constructor/assignment can run.

lvalue — has a name and address (`x` , `arr[0]`).

rvalue — temporary value (`10` , `x+5` , `foo()`).

lvalue reference (&) — binds only to lvalues.

rvalue reference (&&) — binds to rvalues; the basis of move semantics.

Rule of Zero — prefer `std::string` , `std::vector` , `std::unique_ptr` so you never hand-write copy/move/destructor.

OneMarketData follow-up

"Explain what happens in memory when `Test t2 = std::move(t1);` "

Copy Constructor: allocate new resource, copy resource

Move Constructor: steal resource pointer, set old pointer to nullptr, avoid expensiv

Եթե class-ն ունի `int* data;` , առաջին հարցերը՝ Ո՞վ է memory-ի owner-ը: Copy Constructor-ը shallow copy կանխ: Double delete հնարավոր է: Deep copy պետք է: Move Constructor պետք է: Հենց այս մտածողությունն է interviewer-ը փնտրում: